



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és adatszerkezetek II.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2025. szeptember 11.

1 Adattömörítés

- Bevezetés
- Veszteségmentes tömörítés és alapegysége
- Kód, kódfa

2 1 - Betűnkénti kódolás

- 1/A - Naiv módszer
- 1/B - Huffman-kódolás
- Tömörítési algoritmus menete (betömörítés)
- Kódolt szöveg dekódolása (kitömörítés)

3 2 - Szótárkód

- LZW (Lempel-Ziv-Welch)
- Működése és szemléltetése
- Megjegyzések
- LZW kicsomagolás

Adattömörítés

- Informatikában a **kódoláselmélet** adatok különböző reprezentációjával és azok közötti átalakításokkal foglalkozik.
- Ennek egyik ága, a **forráskódolás** az adott alak hosszát vizsgálja; vagyis azt a kérdést, hogy az adott mennyiségű információt mekkora mennyiségű adattal lehet tárolni.
- Legtöbb esetben a cél a rövidebb reprezentáció, tehát beszélhetünk információ- vagy adattömörítésről.

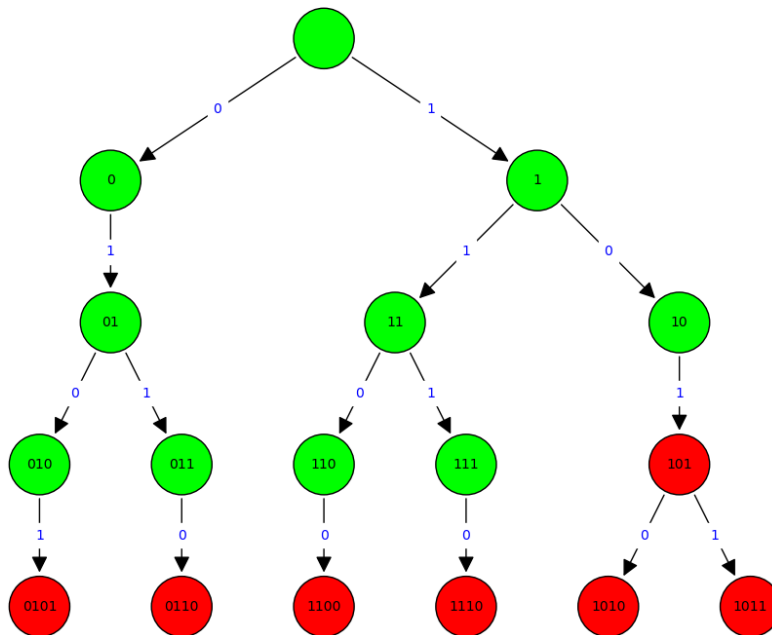
Készült *Nagy Ádám* segédanyaga alapján.

Veszteségmentes tömörítés és alapegysége

- A kódolás során fontos kérdés, hogy az adat teljes egészében visszaállítható-e.
- Tömörítés esetében ennek megfelelően használhatunk **veszteségmentes** vagy veszteséggel járó eljárásokat (pl. JPEG, MPEG, MP3, ...). Mi csak az előbbivel foglalkozunk.
- A kódoláselméletnél meg kell adnunk az **információ alapegységét**, azaz azt, mennyi információtartalma van az atomi „tárolási egységnek”.
- Mivel a jelenlegi számítógépek bináris elven működnek, ez $r = 2$ és így a kódszavaink a $\Gamma = \{ '0', '1' \}$ ábécé feletti szavak lesznek.

- **Kód**nak nevezzük a Γ feletti véges szavak (kódszavak) egy tetszőleges nem üres halmazát.
- Például bináris kód a
 $C = '1011', '1100', '0110', '1110', '1010', '0101', '101'$
- Egy kód szemléletesebb ábrázolásához elkészíthetjük annak **kódfáját**.
- Ebben a fában a fa csúcsai szavak (nem feltétlenül kódszavak), az éleit pedig a kódszavak lehetséges karaktereivel címkézzük.
- A fa gyökerében az üres szó szerepel és egy szóhoz tartozó csúcs leszármazottai azok a szavak, amelyeket úgy kapunk, hogy a szó után írjuk az élen szereplő karaktert.
- A kódhoz tartozó kódfa az a legkevesebb csúcsot tartalmazó ilyen tulajdonságú fa, ami tartalmazza az összes kódszót.

- $C = \text{'1011', '1100', '0110', '1110', '1010', '0101', '101'}$ kód kódfája.
- Kódszavak pirossal vannak jelölve.



- A kódfa szemléltetés mellett más szempontból is hasznos lehet.
- Egyrészt a fa tulajdonságaiból következtethetünk a kód tulajdonságaira, másrészt a kódfa segítségével egy bitsorozat hatékonyan dekódolható:
 - A gyökből indulva a bitek szekvenciájának megfelelően járjuk be a fát, az élek mentén kódszavat keresve és találat esetén ismételve a bejárást megkapjuk a dekódolt adatot.
- Természetesen ez csak akkor igaz, ha a dekódolás egyáltalán lehetséges és egyértelmű.

1 - Betűnkénti kódolás

1 - Betűnkénti kódolás

- A kódolást **betűnkénti kódolás**nak nevezzük, ha az eredeti Σ ábécé feletti adatot betűnként egy $\Sigma \mapsto C \subset \Gamma^*$ kölcsönösen egyértelmű (bijektív) leképezéssel készítjük el.
- Például az **ASCII** kódolás is ilyen, hiszen a megfelelő táblázat alapján betűnként történik a kódolt adat kiszámolása.

ASCII kódtábla: 7biten 128 karakter kódolása

ASCII Code Chart																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	[NUL]	32	20	[SPACE]	64	40	@	96	60	`
1	01	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	02	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	03	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	04	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	05	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	06	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	07	[BELL]	39	27	'	71	47	G	103	67	g
8	08	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	09	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	0A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	0B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	0C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	0D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	0E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	0F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

- Egy kódot **egyenletes kódnak** nevezünk, ha a kód szavainak hossza egyenlő.
- A **naiv módszer** egyenletes kódot használó betűnkénti kódolás.
- A Σ ábécé feletti kódolt adat akkor lesz a legkisebb, ha a kódszavak közös hossza a legkisebb.
- Mivel $|\Gamma| = r$ és $|\Sigma| = d$ ez azt jelenti, hogy az egyes karakterek legkevesebb $\lceil \log_r d \rceil$ hosszal kódolhatóak naiv módszer segítségével.
- Gyakorlatban, ha a tömörítés nem igazán fontos szempont, egyszerűsége miatt sok helyen alkalmazzák, például a 8 bit hosszúságú kódszavakat használó ASCII kód is ilyen.

Naiv módszer példa

Tekintsük a $S = \text{AABCAADEAAB}$ szöveget.

- 1 Naiv módszert alkalmazva hány bittel tudjuk ábrázolni az S -t?
- 2 Adjuk meg S egy kódolt alakját naiv módszer esetén!
- 3 Adjunk meg betűnkénti kódolást, amely tömörebb eredményt ad mint a naiv módszer!
- 4 Adjunk meg egy egyenletes kódolást, amely tömörebb eredményt ad mint a naiv módszer!
- 5 Rajzoljuk fel a korábban használt kód kódfáját! Milyen tulajdonsággal rendelkezik egy egyenletes kódhoz tartozó kódfa?

Naiv módszer példa

- Tekintsük a $S = \text{AABCAADEAAB}$ szöveget.
 - $\Sigma = \text{A,B,C,D,E}$ $|\Sigma|=5$ $\Gamma = 0,1$ $|\Gamma|=2$
 - $\lceil \log_2 5 \rceil = 3$, azaz 3 bites lesz a kód.
- Lehetséges kódtábla:
 - A=000 B=001 C=010 D=011 E=100
- Kódolt szöveg:
 - 000 000 001 010 000 000 011 100 000 000 001
- Kódolt szöveg hossza:
 - $11 \cdot 3 = 33$ bit

1/B - Huffman-kódolás

- Ezt az algoritmust David A. Huffman (1925–1999) írta le először egy mesteri vizsgadolgozatban, és 1952-ben publikálta.
- Intuitíven, betűnkénti kódolás esetén akkor kapunk rövidebb kódolt adatot, ha a gyakori betűkhöz rövid kódszót, a ritkákhoz pedig hosszabbakat rendelünk.
- A **Huffman**-kódolás egy betűnkénti optimális kódolás, azaz az ilyen kódolások között szinte a legjobb tömörítés érhető el vele adott adat esetén.
- A kódolás a Huffman kódfa alapján történik.
- A kódfa felépítése előtt meg kell határozni az ábécé betűinek gyakoriságát a tömörítendő szövegben.
- A kódhoz tartozó kódfát alulról felfelé építjük az eredeti szöveg karaktereinek gyakorisága alapján.

Tömörítési algoritmus menete (betömörítés)

Bináris ($r = 2$) esetben a következőképpen járunk el:

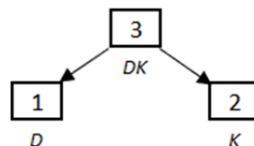
- ① Olvassuk végig a szöveget és határozzuk meg az egyes karakterekhez tartozó gyakoriságokat.
- ② Hozzunk létre minden karakterhez egy csúcsot és helyezzük el azokat egy (min) prioritásos sorban a gyakoriság mint kulcs segítségével.
- ③
 - ① Vegyünk ki két csúcsot a prioritásos sorból és hozzunk létre számukra egy szülő csúcsot.
 - ② A szülő-gyerek éleket címkézzük nullával és eggyel a gyakoriságnak megfelelő sorrendben.
 - ③ Helyezzük el a szülő csúcsot a prioritásos sorba gyerekei gyakoriságának összegét használva kulcsként.
- ④ Ismételjük meg az előző pontot, ha több mint egy csúcs szerepel a sorban.
- ⑤ Olvassuk ki a karakterekhez tartozó kódszavakat a kódfából.
- ⑥ Olvassuk végig újra a bemenetet, kódoljuk karakterenként.

Példa a Huffman algoritmusra

- Legyen a tömritendő szöveg az $S = \text{ABBRRAKKADABBRA}$
- Az ábécé betűi és gyakoriságuk:
 - A – 5
 - B – 4
 - D – 1
 - K – 2
 - R – 3
- összesen: 15 karakter
- Naív módszert használva ($\lceil \log_2 15 \rceil = 4$) $15 \cdot 4 = 60$ bit lenne a tömörített hossz.

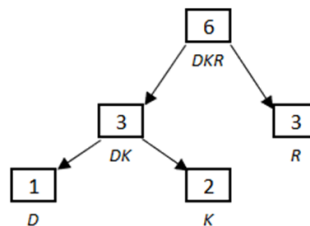
- Rendezzük gyakoriság szerint a betűket, válasszuk a két legkisebb gyakoriságút:

D	1	
K	2	3
R	3	3
B	4	4
A	5	5



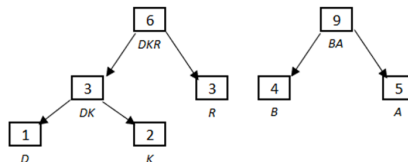
- Válasszuk megint a két legkisebb gyakoriságú csúcsot (az egyik levél, a másik egy belső csúcs)

D	1		
K	2	3	
R	3	3	6
B	4	4	4
A	5	5	5



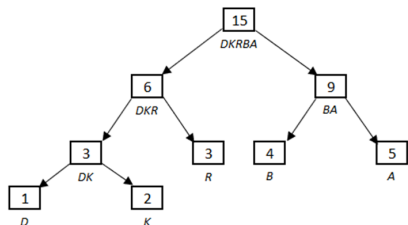
- Így folytatjuk, most a 'B' és 'A' leveleket kötjük össze:

D	1			
K	2	3		
R	3	3	6	6
B	4	4	4	
A	5	5	5	9



- Végül a két belső pontot kötjük össze:

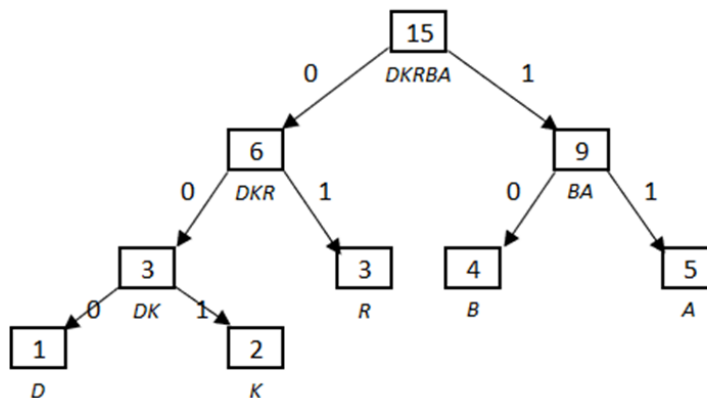
D	1				
K	2	3			
R	3	3	6	6	
B	4	4	4		
A	5	5	5	9	15



- Felcímkezzük az éleket, és kiolvassuk a levélhez vezető úton a címkeket:

Kódtábla

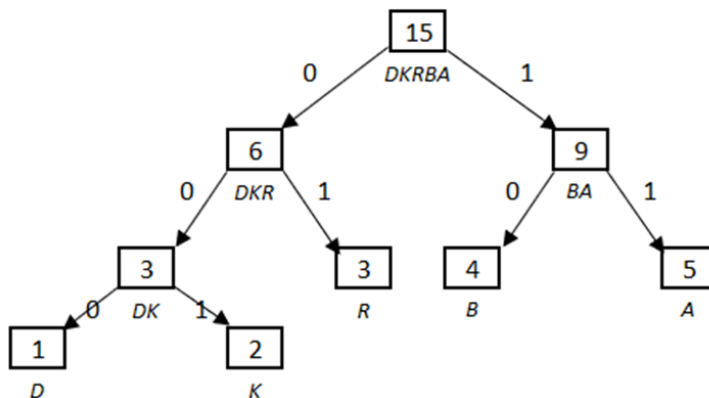
A	11
B	10
D	000
K	001
R	01



- Kódtábla alapján kódoljuk a szöveg betűit, kiírjuk az outputra.

Kódtábla

A	11
B	10
D	000
K	001
R	01



Kódolt szöveg:

A B B R R A K K A D A B B R A
 11 10 10 01 01 11 001 001 11 000 11 10 10 01 11

Kapott kód hosszának kiszámítása

- Egy alkalmas táblázattal könnyen kiszámíthatjuk a kódolt szöveg hosszát.
- Emlékeztetőül, a naív tömörítéssel kapott hossz 45 bit volt.
- Megjegyzés: a tömörített fájl mérete nagyobb lesz, hiszen a kicsomagoláshoz szükséges kódfát is tartalmazza.

Kódolt szöveg hosszának kiszámítása:

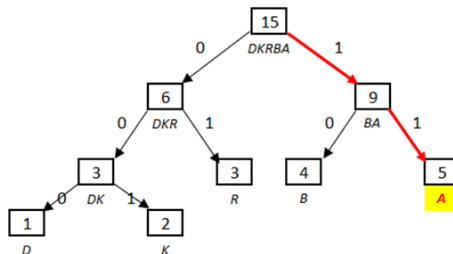
Betű	db	kódhossz	hossz	
A	5	2	10	bit
B	4	2	8	bit
D	1	3	3	bit
K	2	3	6	bit
R	3	2	6	bit
Összesen:			33	bit

Kódolt szöveg dekódolása (kitömörítés)

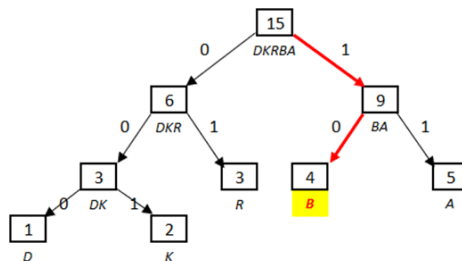
- A dekódoláshoz szükségünk van a kódfára, ezt a kódolt szöveg mellé szokták helyezni.
- A dekódolásnál a kódolt szöveget bitenként dolgozzuk fel az alábbi lépéseket követve:
 - ① Álljunk a kódfa gyökerébe.
 - ② Az olvasott bitek szerint lépegetünk a kódfában balra illetve jobbra, amíg levélhez nem érünk.
 - ③ Levélhez érve olvassuk ki, milyen betűt tartalmaz a levél, és írjuk ki a kimenetre.
 - ④ Ismételjük 1-től, amíg el nem fogynak a kódolt szöveg bitjei.

Kicsomagolás szemléltetése 1/2

- A tömörített bitsorozatot olvasva, mindig a kódfa gyökeréből indulunk, és levélig lépegetünk a bitek szerint:
- 11101001...

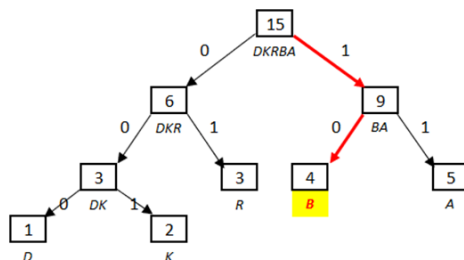


- 11101001...

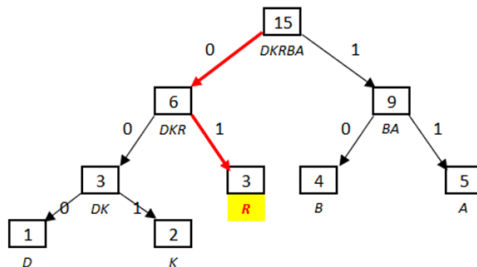


Kicsomagolás szemléltetése 2/2

• 11101001...



• 11101001...



- A Huffman-kód mindig egyértelműen dekódolható (a kódfa segítségével), mivel egy prefix-kód. A **prefix-kód** egy olyan kód, amely esetén a kódszavak halmaza prefixmentes, azaz nincs két olyan kódszó, ami esetén az egyik a másiknak valódi prefixe lenne. Ez a tulajdonság a kódfára azt jelenti, hogy minden kódszóhoz tartozó csúcs levél.
- Általában a Huffman-kódolás nem egyértelmű. Egyrészt ha több azonos gyakoriság van, akkor bármelyiket választva Huffman-kódolást kapunk; másrészt a 0 és 1 szerepe felcserélhető.
- Mivel a kódolás minden adathoz különböző, a dekódoló oldalon is ismertnek kell lennie. Ez gyakorlatban azt jelenti, hogy
 - a kódfát vagy kódtáblát is csatolnunk kell a kódolt adathoz (ront a tömörítési arányon), vagy
 - a Huffman-kódot általánosított adathoz készítjük el. Például magyar szöveg kódolásánál a magyar nyelv karaktereinek általános gyakorisága alapján (ált. tömörít, de nem az optimális kód).

- Készíts rekurzív algoritmust, mely a Huffman kódfát bejárva, elkészíti a kódtáblát. A tavaly tanult, megfelelő bináris fa rekurzív bejáró algoritmusra támaszkodva járjuk be a fát, és leveleknél írjuk ki a szelektort. A szelektor előállításához egy paraméterként megadott verem, vagy string használható.

2 - Szótárkód

Szótárkód: LZW (Lempel-Ziv-Welch)

- Az LZW elnevezés Lempel-Ziv-Welch szerzőhármassal nevének kezdőbetűiből származik. Az informatikában széles körben használt eljárást Terry Welch publikálta 1984-ben az Abraham Lempel és Jacob Ziv által 1978-ban közzétett LZ78 algoritmus továbbfejlesztéseként.
- A betűnkénti kódolás tömörítési tulajdonsága ismert és elmondható, hogy hatékonysága korlátozott is. Könnyen tudunk olyan adatot adni, amit sokkal tömörebben formában lehet reprezentálni, ha a kódolás nem karakterenként történik. Ezt az észrevételt használják ki a **szótárkódok** úgy, hogy egy kódszó nem csak egy karakter képe lehet, hanem egy szóé is.
- Az **LZW kódolás** egy kezdeti kódtáblát bővít lépésről-lépésre úgy, hogy egyre hosszabb már „látott” szavakhoz rendel új kódszót. Ezzel a valós adatok azt a tulajdonságát használjuk ki, hogy abban relatív rövid részek sűrűn ismétlődnek. Például gondoljunk élő nyelvben milyen sűrűn fordulnak elő névelők, kötőszavak, stb.

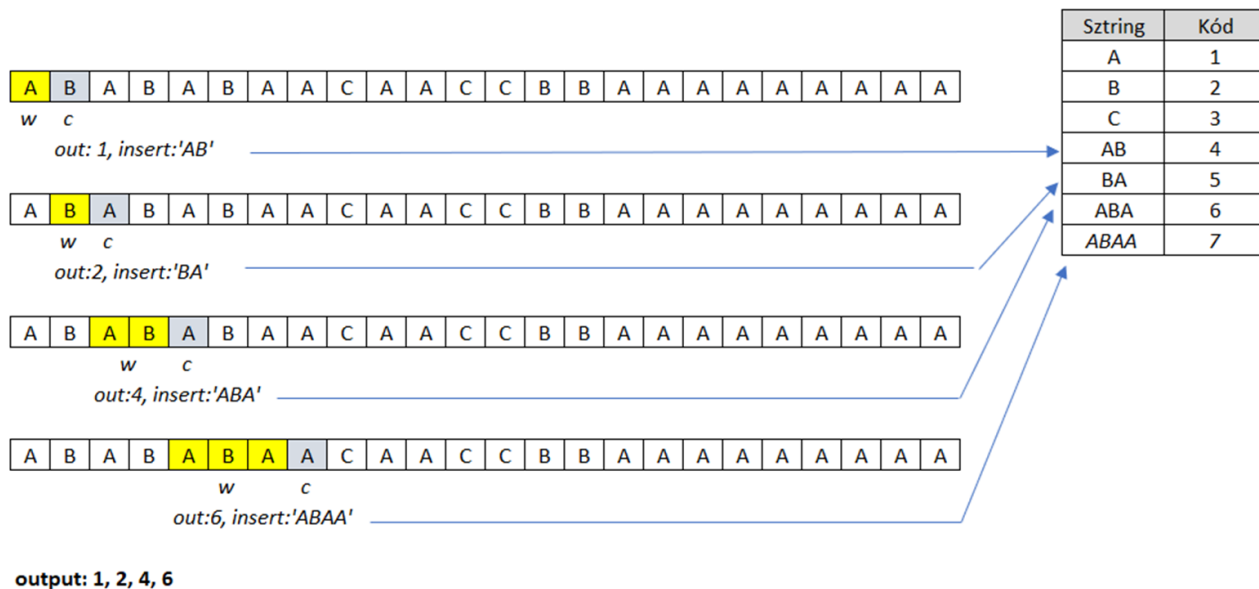
LZW működése és szemléltetése

- Legyen $S = \text{ABABABAACAACCBBAAAAAAAAAA}$ a példaszövegünk és így a kezdeti kódtáblánk
- Induló szótárunk az szöveg ábécéjét tartalmazza:

Sztring	Kód
A	1
B	2
C	3
...	

- A bemenetet pontosan egyszer fogjuk olvasni a kódolás során úgy, hogy mindig a már ismert (kóddal rendelkező) leghosszabb következő szót keressük. Ha megtaláltuk, akkor egyrészt
 - kiírjuk a talált szó kódját a kimenetre (eredmény), másrészt
 - bővítjük a kódszavak halmazát az αc szó képével, ahol az α a talált szó és c a következő karakter.

LZW működése és szemléltetése 1/5



LZW működése és szemléltetése 2/5

A B A B A B A **A** C A A C C B B A A A A A A A A A

w c

out:1, insert:'AC'

A B A B A B A A **C** A A C C B B A A A A A A A A A

w c

out:3, insert:'CA'

A B A B A B A A C **A** A C C B B A A A A A A A A A

w c

out:1, insert:'AA'

A B A B A B A A C A **A** **C** C B B A A A A A A A A A

w c

out:8, insert:'ACC'

A B A B A B A A C A A C **C** B B A A A A A A A A A

w c

out:3, insert:'CB'

output: 1, 2, 4, 6, 1, 3, 1, 8, 3

Sztring	Kód
A	1
B	2
C	3
AB	4
BA	5
ABA	6
ABAA	7
AC	8
CA	9
AA	10
ACC	11
CB	12

LZW működése és szemléltetése 3/5

A B A B A B A A C A A C C B B A A A A A A A A A A

w c
out:2, insert:'BB'

A B A B A B A A C A A C C B B A A A A A A A A A A

w c
out:5, insert:'BAA'

A B A B A B A A C A A C C B B A A A A A A A A A A

w c
out:10, insert:'AAA'

A B A B A B A A C A A C C B B A A A A A A A A A A

w c
out:15, insert:'AAAA'

A B A B A B A A C A A C C B B A A A A A A A A A A

w
out:15

output: 1, 2, 4, 6, 1, 3, 1, 8, 3, 2, 5, 10, 15, 15

Sztring	Kód
A	1
B	2
C	3
AB	4
BA	5
ABA	6
ABAA	7
AC	8
CA	9
AA	10
ACC	11
CB	12
BB	13
BAA	14
AAA	15
AAAA	16

LZW működése és szemléltetése 4/5

A B A B A B A A C A A C C B B A A A A A A A A A

Tömörebb lejátszás

Kód	Aktuális szó	Következő betű	Új kód
1	A	B	4
2	B	A	5
4	AB	A	6
6	ABA	A	7
1	A	C	8
3	C	A	9
1	A	A	10
8	AC	C	11
3	C	B	12
2	B	B	13
5	BA	A	14
10	AA	A	15
15	AAA	A	16
15	AAA	---	---

▲
output

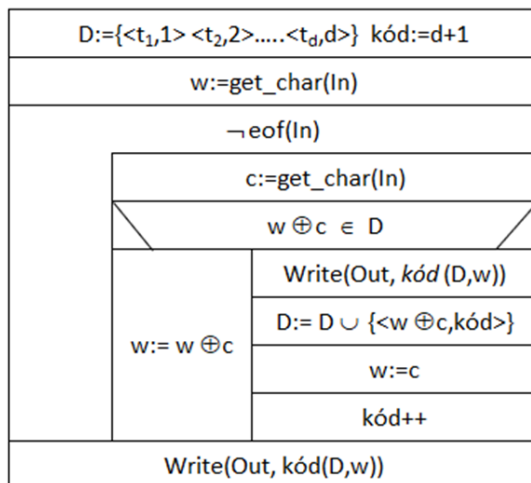
Szting	Kód
A	1
B	2
C	3
AB	4
BA	5
ABA	6
ABAA	7
AC	8
CA	9
AA	10
ACC	11
CB	12
BB	13
BAA	14
AAA	15
AAA	16

Induló
szótár

szótárba felvett szavak

LZW működése és szemléltetése 5/5

LZW_Compress(In,Out, Σ)



In tömörítendő szöveg
 Out kimenet
 Σ Input ábécé, betűi: t₁, ..., t_d
 D szótár, <sztring, kód> párokból áll
 w aktuális szó (ami benne van már a szótárban)
 c utolsónak olvasott betű

Kezdeti szótár létrehozása, kód segéd változó.

Egy betű olvasása, ez lesz az aktuális szó.

Van még az inputon betű?

Az inputról olvassuk a következő betűt.

w és c konkatenációja szerepel a szótárban?

ha igen, bővítjük az aktuális szót az utolónak beolvasott betűvel

ha nem, az aktuális szó kódját kiírjuk, felvesszük

w és c konkatenációját a szótárba,

az utolsónak olvasott betű lesz az aktuális szó.

- Fontos észrevenni, hogy egy hosszú szöveg esetén az ismertetett eljárás annyi új kódszót is bevezethet, hogy az azok közötti keresés összemérhető lenne a teljes szöveg végigolvasásával. Természetesen ezt nem szeretnénk, ezért gyakorlatban korlátozzuk a kódszavak halmazát. Ez történhet például:
 - a kódszavak számának korlátozásával;
 - a kódszavakhoz tartozó szavak hosszának korlátozásával;
 - azzal, hogy a bemenet csak egy kezdőszeletén építjük a szótárat, utána csak kódolunk.
- Mivel a Huffman-kódolás csak a betűnkénti kódolások között optimális az LZW eljárás könnyen eredményezhet rövidebb kódolt alakot, annak ellenére is, hogy az itt használt kódszavakat még binárisan kódolni kell.
- Az LZW eljárás egyszerűnek nevezhető (összehasonlítva például a Huffman kódolással) és mivel csak egyszer kell olvasni a bemenetet, hatékony is (amennyiben a kódszavak tárolása hatékony).

- 1 Szemléltessük a ABCABCABCAAABBCCAABABA kódolását.
- 2 Dekódoljuk az 1,3,4,5,6 üzenetet, ha a kezdeti tábla $A \rightarrow 1$ és $B \rightarrow 2$.
- 3 Adjunk meg olyan 20 hosszú három karaktert tartalmazó (legalább egyszer mindegyiket) szöveget, ami esetén a kódolt (LZW) szöveg pontosan 13 hosszú.
- 4 Keressünk olyan szöveget, aminek LZW kódolása rövidebb eredményt ad a Huffman-kódolással összehasonlítva.

LZW kicsomagolás - Pazarló

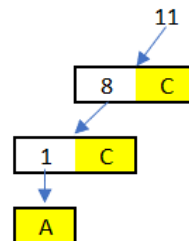
- Kicsomagolás első lehetséges módja, hogy a teljes szótárt beletesszük a csomagba. Mivel a szótárban szereplő sztringek igen hosszúak is lehetnek, ez nagyon megnöveli a csomag méretét.
- Welch ötlete, a szótártömörítésére (csak érdekesség, nem vizsgaanyag):
- Végül megszületett a kicsomagoló algoritmus, melynek csak a kezdeti szótárra van szüksége!

Szting	Kód
A	1
B	2
C	3
AB	4
BA	5
ABA	6
ABAA	7
AC	8
CA	9
AA	10
ACC	11
CB	12
BB	13
BAA	14
AAA	15
AAAA	16



Szting	Kód
A	1
B	2
C	3
1B	4
2A	5
4A	6
6A	7
1C	8
3A	9
1A	10
8C	11
3B	12
2B	13
4A	14
10A	15
15A	16

például a 11 dekódolása:



kapott szó: ACC

a szavak az ismert prefix kódjából és az utolsó betűből állnak

LZW kicsomagolás - csak az induló szótár segítségével

- Az induló szótár rendelkezésre áll.

Sztring	Kód
A	1
B	2
C	3
...	

- Ötlet: kicsomagolás közben rekonstruáljuk a szótárt!
- Azaz végezzük a „kódolás” lépéseit is: az aktuális kódnak megfelelő szóhoz hozzáragasztjuk a következő kód első betűjét, az így kapott szó lesz a szótár következő szava.
- Ezzel a módszerrel dekódolhatjuk a szöveget, és közben az eredeti szótárt is újraépítjük!

LZW kicsomagolás - csak az induló szótár segítségével

tömörítés outputja: 1, 2, 4, 6, 1, 3, 1, 8, 3, 2, 5, 10, 15, 15

Sztring	Kód
A	1
B	2
C	3
AB	4
BA	5

aktuális kód

1

A

következő kód

2

B

AB

aktuális kód

2

B

következő kód

4

AB

BA

aktuális kód

4

AB

következő kód

6

?

- Az első két lépés működik, de a harmadik lépésnél látszólag elakadunk!
- 4 lesz az aktuális kód, 6 pedig a következő, de még nincs 6-os kódú szó a szótárunkban!
- De ki lehet találni!

LZW kicsomagolás - csak az induló szótár segítségével

tömörítés outputja: 1, 2, 4, 6, 1, 3, 1, 8, 3, 2, 5, 10, 15, 15

Sztring	Kód
A	1
B	2
C	3
AB	4
BA	5

aktuális kód

következő kód

4

6

AB

?

Még nincs,

de azt tudjuk, hogy úgy keletkezett, hogy a 4-es, azaz az AB szóhoz ragasztottunk egy betűt
Tehát 'A'-val fog kezdődni!

4

6

AB

A*

Tehát a 6-os kód az:

ABA

aktuális kód

következő kód

6

1

ABA

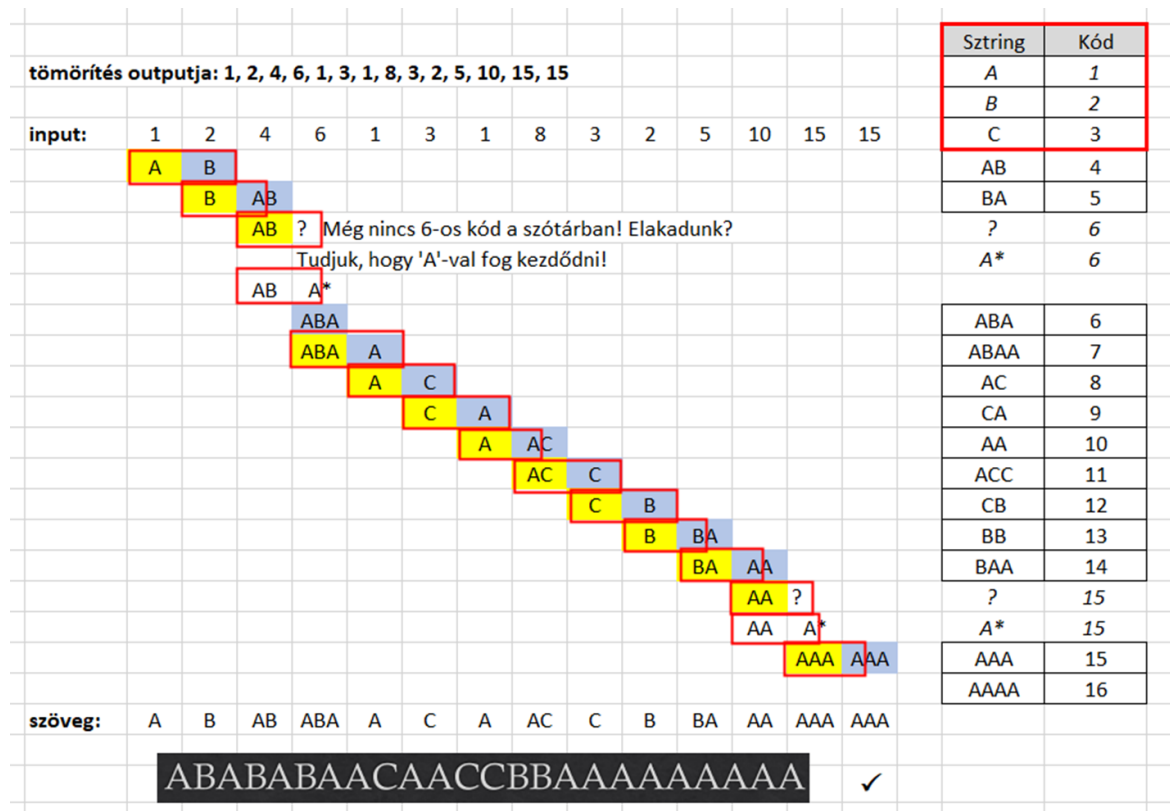
A

ABAA

Sztring	Kód
A	1
B	2
C	3
AB	4
BA	5
ABA	6
ABAA	7

- Az új szótári szó első betűjét már ismerjük, és az elég.
- Az aktuális kódszó első betűjét önmagához fűzzük, így megkapjuk a még ismeretlen szót.
- Ezt aztán a szótárba is felvesszük, illetve a dekódolt szövegbe is beírjuk.
- és folytatódik az algoritmus...

LZW kicsomagolás - csak az induló szótár segítségével



LZW kicsomagolás - csak az induló szótár segítségével

LZW_Decompress(In, Out, Σ)

D := {<t ₁ , 1> <t ₂ , 2> <t _d , d>} kód := d+1													
k := get_code(In)													
s := string(D, k)													
Write(Out, s)													
¬ eof(In)													
<table> <tr> <td colspan="2">k := get_code(In)</td></tr> <tr> <td colspan="2">k < kód</td></tr> <tr> <td>w := string(D, k)</td><td>w := s ⊕ s₁</td></tr> <tr> <td>Write(Out, w)</td><td>Write(Out, w)</td></tr> <tr> <td>D := D ∪ {<s ⊕ w₁, kód>}</td><td>D := D ∪ {<w, kód>}</td></tr> <tr> <td colspan="2">s := w ++ kód</td></tr> </table>		k := get_code(In)		k < kód		w := string(D, k)	w := s ⊕ s ₁	Write(Out, w)	Write(Out, w)	D := D ∪ {<s ⊕ w ₁ , kód>}	D := D ∪ {<w, kód>}	s := w ++ kód	
k := get_code(In)													
k < kód													
w := string(D, k)	w := s ⊕ s ₁												
Write(Out, w)	Write(Out, w)												
D := D ∪ {<s ⊕ w ₁ , kód>}	D := D ∪ {<w, kód>}												
s := w ++ kód													

In	kódsorozat, a tömörített állomány
Out	a visszaállított eredeti szöveg
Σ	Input ábécé, betűi: t ₁ , ..., t _d
D	szótár, <sztring, kód> párokból áll
k	aktuális kód
s	aktuális kódnak megfelelő sztring, s ₁ - s első betűje
w	következő kódnak megfelelő sztring, w ₁ - az első betűje

Kezdeti szótár létrehozása, kód segéd változó.

Első kód beolvasása, ez biztosan szerepel a szótárban.

A kódhoz tartozó szöveg (szótár indexelésével meghatározható)

A kapott s szó kiírása a kimenetre.

Van még az inputon kód?

Az inputról olvassuk a következő kódot.

Ha k kisebb, mint kód, akkor a szó már szerepel a szótárban.

ha igen: a szót kiírjuk, majd s és w₁ konkatenációját felvesszük a szótárba.

ha nem szerepel, ki lehet találni: s és első betűjének konkatenáltja

a keresett szó. Kiírjuk a kimenetre, hozzávesszük a szótárhoz.

w lesz az aktuális kódnak megfelelő sztring, a szótár mindenképp egy új szóval bővült, így kód-ot növeljük eggyel.

- Egy szöveget LZW algoritmussal tömörítettünk. A következő kódsorozatot kaptuk:
- 3,1,2,5,4,6,9,7,11,2
- Az eredeti szótár $A=1$, $B=2$, $C=3$ szavakat tartalmazta.
- Mutassa be a dekódoló algoritmus működését:
- Fejtse meg, mi volt a tömörítendő szöveg,
- Rekonstruálja az algoritmus által létrehozott szótárt!

Köszönöm a figyelmet!